

Day 2 – Android Static Analysis Assignment

JADX, APKTool and MobSF – Standard Practical Tasks

Based on: Static Analysis Android Application Lab Manual

Platform: Windows or Ubuntu Linux

Mode: Individual practical assignment

Target: Instructor-provided APK (e.g., AndroGoat_patched) or another instructor-approved training APK

Instructions

1. Complete this assignment using the Static Analysis Android Application Lab Manual as your primary reference. The manual section for each task is given in its Hint.
2. Use only the APK approved by the instructor. Do not analyze unauthorized applications.
3. Record the APK filename, file size, SHA-256 hash, and analysis date before starting.
4. Record every command, tool step, output, and observation in a single text file. Take a screenshot wherever a task asks for one, and redact passwords, tokens, personal data, or private URLs.
5. Do not report a JADX search hit or MobSF alert as a confirmed vulnerability until the referenced code, manifest entry, or resource file has been manually verified.
6. If a step fails or a tool behaves unexpectedly, record the exact error/behavior and the corrective step you attempted, referencing the manual section used.
7. Convert the completed text file into PDF and upload it to the LMS before the deadline.

Part A: Manual Static Analysis – JADX and APKTool

Objective: Identify the target APK, decompile and inspect its source in JADX, search for security-relevant code, decode the APK with APKTool, and review its manifest, deep links/WebViews, and resource files for security issues.

Manual sections to use: Sections 5.1 – 5.7 of the Static Analysis Android Lab Manual.

- **Task 1** — Open the target APK in JADX-GUI, decompile it, and identify its package name, main/login activity, and one other security-relevant class.

***Hint:** Launch jadx-gui, use Open file, then expand the package tree. Look for authentication, WebView, network, or database helper classes. Refer to Section 5.2. Screenshot required.*

- **Task 2** — Search the decompiled project for security-relevant keywords and identify two or three genuinely relevant results.

***Hint:** Use **Ctrl+Shift+F** in JADX and search keywords one at a time (e.g., password, token, secret, WebView, addJavascriptInterface, AES, SharedPreferences, root, Debuggable). Refer to Section 5.3. For each hit record the class, method, and why it is security-relevant.*

- **Task 3**— Decode the same APK using APKTool and confirm the expected output structure was produced.
Hint: Run `apktool d <target.apk> -o <decoded_folder>` from the folder containing the APK. Refer to Section 5.4. Confirm `AndroidManifest.xml`, `res/`, and `smali/` are present.
- **Task 4**— Review the decoded `AndroidManifest.xml` for risky configuration flags, list its components, and identify its declared permissions.
Hint: Check `android:debuggable`, `android:allowBackup`, `android:usesCleartextTraffic`, and `android:networkSecurityConfig`; list `<activity>`/`<service>`/`<receiver>`/`<provider>` entries and whether each is exported. Refer to Section 5.5. Write 'Not observed' for anything missing.
- **Task 5**— Check the manifest and JADX for custom URI schemes/deep links and risky `WebView` configuration.
Hint: Search the manifest for `<intent-filter>`, `<data>`, `android:scheme/host/path`, and search JADX for `loadUrl`, `setJavaScriptEnabled(true)`, and `addJavascriptInterface`. Refer to Section 5.6.
- **Task 6** — Review the decoded resource and asset files for hardcoded sensitive values.
Hint: Search `shop_decoded/res/values/strings.xml`, `res/xml/`, `res/raw/`, and `assets/` for keywords such as `password`, `token`, `secret`, `api_key`, `server URL`, or `encryption key`. Refer to Section 5.7. Write 'No matching output observed' if nothing is found.

Part B: Automated Cross-Check – MobSF

Objective: Run the same APK through MobSF and cross-verify its automated findings against the evidence you gathered manually in Part A.

Manual sections to use: Sections 5.8 and 5.9 of the Static Analysis Android Lab Manual.

- **Task 8** — Scan the target APK using the approved MobSF instance and record its overall security score along with one manifest-related and one code-related finding.
Hint: Use Upload & Analyze, wait for the scan to complete, then open View Report. Refer to Section 5.8. Review the Application Information, Security Score, Manifest Analysis, and Code Analysis sections.
- **Task 9** — Compare two MobSF findings against your own manual evidence from Part A and mark each as verified, not verified, or a false positive.
Hint: For each MobSF alert, locate the same code, manifest entry, or resource file yourself before accepting it. Refer to Sections 5.8 – 5.9. Do not report an alert you have not personally verified.

Deliverables

- Completed APK identification table (filename, size, SHA-256 hash, analysis date).
- All commands/steps executed and their outputs, recorded in the order performed, in a single text file.
- All screenshots requested in Part A and Part B, numbered and referenced in the text, with sensitive data redacted.
- Completed tables for JADX keyword search, manifest review, and MobSF cross-verification.
- A short conclusion of 120–150 words describing the most important security-relevant observation.
- Convert the completed text file into PDF and upload it to the LMS before the deadline.